

Final Project: Create A Time Tracking App

Estimated time: **60** minutes



Introduction

In this final project, you will build a time tracking app using Flutter. This app will allow users to track the time spent on various tasks and projects. The data will be saved locally, ensuring that entries are not lost when the app is closed and reopened.

User stories

The following user stories outline the key functionalities that you need to implement:

- “I want to view a list of time spent on different tasks to manage my activities efficiently.”
- “I want to add a time tracking entry with fields for project, task, total time, date, and notes to record detailed information about my activities.”
- “I want my entries to be saved locally to preserve them across app sessions.”
- “I want to group my time by projects to understand better how much time I spend on each project.”
- “I want to delete a time entry to remove any incorrect or unnecessary data.”
- “I want to manage projects and tasks in the app settings.”

Considerations

This application is very similar to the practice application you worked on previously. Feel free to use the same architecture.

- **lib/models:** Contains the model files.
- **lib/provider:** Contains the providers that back up the widgets.
- **lib/dialogs:** Contains widgets that get used in a screen.
- **lib/screens:** Contains different screens for adding time and managing projects and tasks.

While we don't provide you with every line of code, you should be able to create this application by revisiting the practice project, which provided all the code.

Saving Your Progress: How to Download Your Project

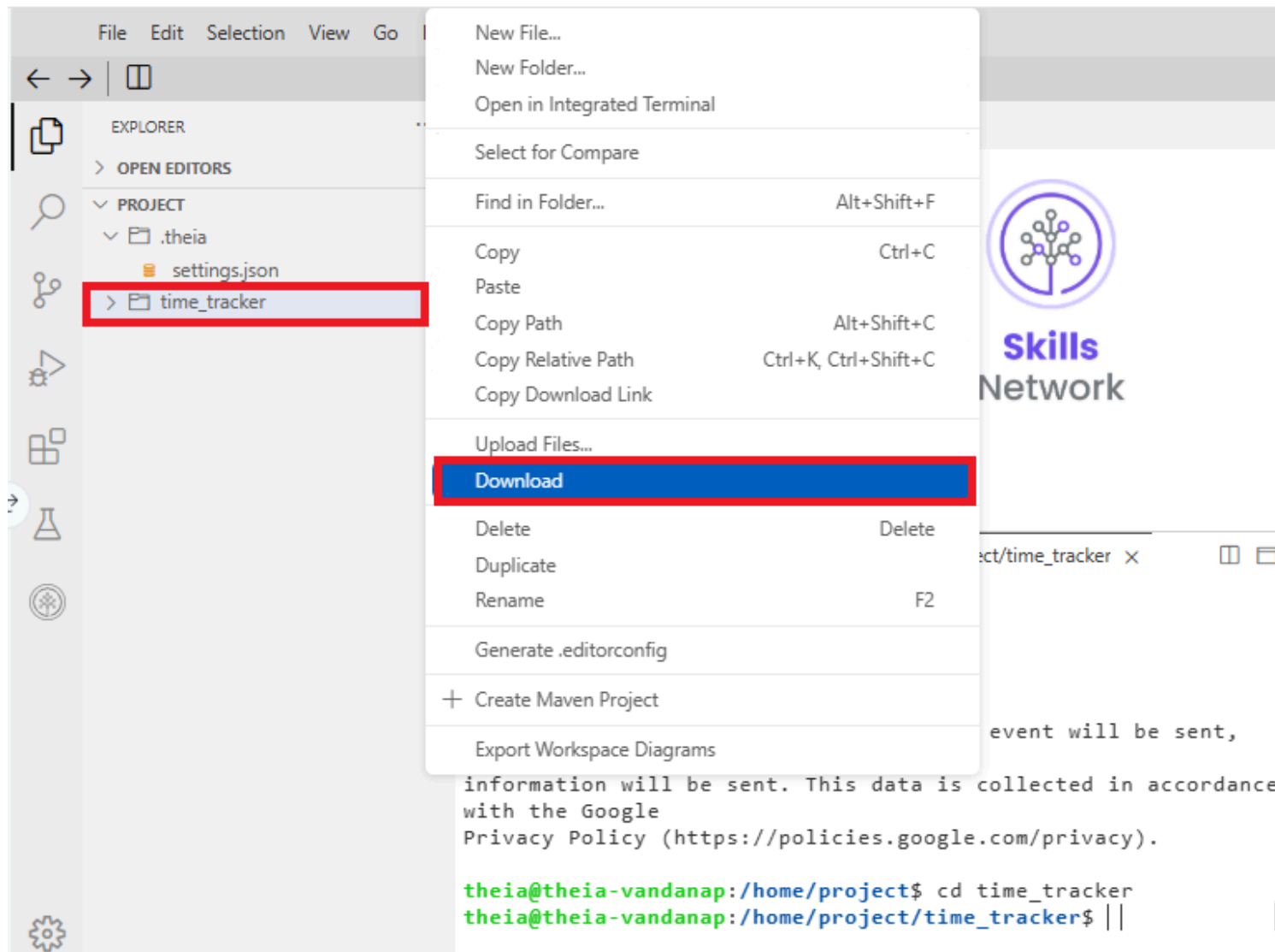
Understand Session-Based Storage: Keep in mind that our lab environments are session-based and do not retain data from previous sessions. Downloading your project ensures that your work is preserved.

Download Your Project: To avoid losing any progress, download your project by following these steps:

Right-click on the project folder and locate the download button.

Click the download button to save a copy of your project to your local machine.

Before ending your session, verify that all changes to your project are saved.



Creation of a Time Tracking App

Note:

1. Please ensure that all updates to the files are saved properly.

2. Download your project files and capture screenshots as requested in the project.
3. As mentioned in the "Final Project: Submission and Grading Criteria", you can submit your project deliverables through either **Option 1: AI-Graded Submission and Evaluation** or **Option 2: Peer-Graded Submission and Evaluation**.
4. **For Option 1: AI-Graded Submission and Evaluation:**
Submission requires full and updated code of home_screen.dart, add_time_entry_screen.dart, project_management_screen.dart, task_management_screen.dart and time_entry_provider.dart.
5. **For Option 2: Peer-Graded Submission and Evaluation:**
Submission requires screenshots as mentioned under heading **Screenshots for the peer review**. You can take these screenshots once you complete the project

Step 1: Creation of the Flutter project

1. Create the flutter project in the terminal.

```
flutter create time_tracker  
cd time_tracker
```

2. Add dependencies to the project.

```
flutter pub add provider  
flutter pub add localstorage  
flutter pub add intl  
flutter pub add collection
```

Let's start adding code now. We give you partial code and files. You can add additional supporting files as needed in order to complete all features.

Step 2: Add models

Next, you can work on models for the project. You will need to define models for Project, Task, and Time Entry. Here is some code to get you started. You can split them into multiple files for better organization.

```
class Project {
    final String id;
    final String name;
    Project({required this.id, required this.name});
}
class Task {
    final String id;
    final String name;
    Task({required this.id, required this.name});
}
class TimeEntry {
    final String id;
    final String projectId;
    final String taskId;
    final double totalTime;
    final DateTime date;
    final String notes;
    TimeEntry({
        required this.id,
        required this.projectId,
        required this.taskId,
        required this.totalTime,
        required this.date,
        required this.notes,
    });
}
```

Step 3: Add a provider

Set up a provider to manage state-related time entries. The entries should be stored in local storage just as you did for the practice project. Name this file as `time_entry_provider.dart` in the `lib/provider` folder.

```
class TimeEntryProvider with ChangeNotifier {  
  List<TimeEntry> _entries = [];  
  List<TimeEntry> get entries => _entries;  
  void addTimeEntry(TimeEntry entry) {  
    _entries.add(entry);  
    notifyListeners();  
  }  
  void deleteTimeEntry(String id) {  
    _entries.removeWhere((entry) => entry.id == id);  
    notifyListeners();  
  }  
}
```

Note: As in the practice project, expenses, categories, and tags were managed in one provider; similarly, in this project, we will manage projects, tasks, and time entries in a single provider.

Step 4: Add screens

Let's work on the screens. This code here will get you started, but feel free to change colors, styles, and widgets as long as you meet the user stories.

Home screen

This screen displays all time entries. Each list item details the time spent on tasks and allows the user to interact with each entry. We are just showing a single screen in this code, but your code should contain two tabs. First, for the list of entries, and second, for entries grouped by project. This is similar to the expenses grouped by category in the practice project. Feel free to revisit that code to get some ideas.

Keep the name as `home_screen.dart` in the `lib/screens` folder

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'models/time_entry.dart';
import 'providers/time_entry_provider.dart';
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Time Entries'),
      ),
      body: Consumer<TimeEntryProvider>(
        builder: (context, provider, child) {
          return ListView.builder(
            itemCount: provider.entries.length,
            itemBuilder: (context, index) {
              final entry = provider.entries[index];
              return ListTile(
                title: Text('${entry.projectId} - ${entry.totalTime} hours'),
                subtitle: Text('${entry.date.toString()} - Notes: ${entry.notes}'),
                onTap: () {
                  // This could open a detailed view or edit screen
                },
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Navigate to the screen to add a new time entry
          Navigator.push(
            context,
            MaterialPageRoute(builder: (context) => AddTimeEntryScreen()),
          );
        },
        child: Icon(Icons.add),
        tooltip: 'Add Time Entry',
      ),
    );
  }
}
```

Add time entry screen

This screen provides a form to add a new time entry. Make sure you are using the following widgets for each field:

- project: dropdown
- task: dropdown
- time: integer
- date: date widget
- notes: free form text

The code below will get you started. Notice that it has the list of projects and lists hardcoded. That is fine for this project. Ideally, you would store these lists in a database.

Keep the name as `add_time_entry_screen.dart` in the `lib/screens` folder

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'models/time_entry.dart';
import 'providers/time_entry_provider.dart';
class AddTimeEntryScreen extends StatefulWidget {
  @override
  _AddTimeEntryScreenState createState() => _AddTimeEntryScreenState();
}
class _AddTimeEntryScreenState extends State<AddTimeEntryScreen> {
  final _formKey = GlobalKey<FormState>();
  String projectId = '';
  String taskId = '';
  double totalTime = 0.0;
  DateTime date = DateTime.now();
  String notes = '';
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Add Time Entry'),
```



```

),
body: Form(
  key: _formKey,
  child: Column(
    children: <Widget>[
      DropdownButtonFormField<String>(
        value: projectId,
        onChanged: (String? newValue) {
          setState(() {
            projectId = newValue!;
          });
        },
        decoration: InputDecoration(labelText: 'Project'),
        items: <String>['Project 1', 'Project 2', 'Project 3'] // Dummy project names
          .map<DropdownMenuItem<String>>((String value) {
            return DropdownMenuItem<String>(
              value: value,
              child: Text(value),
            );
          }).toList(),
      ),
      DropdownButtonFormField<String>(
        value: taskId,
        onChanged: (String? newValue) {
          setState(() {
            taskId = newValue!;
          });
        },
        decoration: InputDecoration(labelText: 'Task'),
        items: <String>['Task 1', 'Task 2', 'Task 3'] // Dummy task names
          .map<DropdownMenuItem<String>>((String value) {
            return DropdownMenuItem<String>(
              value: value,
              child: Text(value),
            );
          }).toList(),
      ),
      TextFormField(
        decoration: InputDecoration(labelText: 'Total Time (hours)'),
        keyboardType: TextInputType.numberWithOptions(decimal: true),
        validator: (value) {
          if (value == null || value.isEmpty) {
            return 'Please enter total time';
          }
          if (double.tryParse(value) == null) {
            return 'Please enter a valid number';
          }
          return null;
        },
      ),
    ],
  ),
),

```

```

        onSave: (value) => totalTime = double.parse(value!),
      ),
      TextFormField(
        decoration: InputDecoration(labelText: 'Notes'),
        validator: (value) {
          if (value == null || value.isEmpty) {
            return 'Please enter some notes';
          }
          return null;
        },
        onSave: (value) => notes = value!,
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            _formKey.currentState!.save();
            Provider.of<TimeEntryProvider>(context, listen: false)
              .addTimeEntry(TimeEntry(
                id: DateTime.now().toString(), // Simple ID generation
                projectId: projectId,
                taskId: taskId,
                totalTime: totalTime,
                date: date,
                notes: notes,
              ));
            Navigator.pop(context);
          }
        },
        child: Text('Save'),
      ),
    ],
  ),
),
);
}
}
}

```

- **DropDownButtonFormField:** This widget is used to create dropdowns for the project and task fields. You will need to replace the dummy values ("Project 1", "Project 2", and so on) with actual data from your app's state or database.
- **TextFormField for Total Time:** We've added `TextInputType.numberWithOptions(decimal: true)` to allow decimal inputs, suitable for entering time in hours with precision.
- **Validators:** Each form field has a validator to ensure that the user inputs valid and required data. This will help prevent submission errors and enforce data integrity.

Project and task management screen

These screens allows users to manage projects and tasks.

Keep the name as `task_management_screen.dart` in the `lib/screens` folder

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/time_entry_provider.dart';
import '../widgets/add_task_dialog.dart';
import '../models/task.dart';
class TaskManagementScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Manage Tasks'),
      ),
      body: Consumer<TimeEntryProvider>(
        builder: (context, provider, child) {
          // Lists for managing tasks would be implemented here
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Add new task
        },
        child: Icon(Icons.add),
        tooltip: 'Add Task',
      ),
    );
  }
}
```

Keep the name as `project_management_screen.dart` in the `lib/screens` folder

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/time_entry_provider.dart';
import '../widgets/add_project_dialog.dart';
import '../models/project.dart';
class ProjectManagementScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Manage Projects'),
      ),
      body: Consumer<TimeEntryProvider>(
        builder: (context, provider, child) {
          // Lists for managing projects would be implemented here
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Add new project
        },
        child: Icon(Icons.add),
        tooltip: 'Add Project',
      ),
    );
  }
}
```

Step 5: Add Widgets

In this step, you will create the necessary widgets for the application. You'll create an AddProjectDialog widget and an AddTaskDialog widget to handle adding new project and tasks.

Part 1: Create the AddProjectDialog widget

Keep the file name as `add_project_dialog.dart`

- Pop-up dialog to input a **project name**.
- Displays "**Add Project**" as the title and "**Project Name**" as the input label.
- **Cancel** closes the dialog; **Add** returns the entered name
- Properly disposes the TextEditingController.

Part 2: Create the AddTaskDialog widget

Keep the file name as `add_task_dialog.dart`

- Pop-up dialog to input a **task name**.
- Displays "**Add Task**" as the title and "**Task Name**" as the input label.
- **Cancel** closes the dialog; **Add** returns the entered task name
- Properly disposes the TextEditingController.

These templates have been provided as a reference. Feel free to revisit the practice project to implement these features.

Assessment:

Option 1: AI-Graded Submission and Evaluation

Please make sure to download and keep a backup of your project files. For the final evaluation, you must copy/paste the complete and updated versions of the following code files:

- `home_screen.dart`
- `add_time_entry_screen.dart`
- `project_management_screen.dart`
- `task_management_screen.dart`
- `time_entry_provider.dart`

Ensure that all submitted code reflect your latest upadates.

Step 5: Run the lab

Follow these steps to run the application in the Cloud IDE.

Flutter provides `hot reload` that makes it very easy to test the code as you are developing it. That functionality however does not work in the Cloud IDE. We have developed a `hot_reload.sh` script that essentially does the same thing. Please follow these steps to run the flutter application:

1. Change to the home directory

```
cd /home/project
```

2. Set the `PROJECT_DIR` variable. This should be set to the `lib` directory of your flutter app folder.

```
export PROJECT_DIR=/home/project/time_tracker/lib
```

3. Get the script and save to the `/home/project` folder.

```
wget -O /home/project/hot_reload.sh https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/S3cw83zxbxBhfixSSMK1Uw/hot-reload.sh
```

4. Make the script executable by using the `chmod` command.

```
chmod +x ./hot_reload.sh
```

5. Run the command.

```
./hot_reload.sh
```

You should see output like this:

```
theia@theia-captainfedo1:/home/project$ ./hot_reload.sh
Using PROJECT_DIR: /home/project/time_tracker/lib
inotify-tools is already installed.
lsof is already installed.
Starting Flutter web server...
Port 8080 is free.
Compiling lib/main.dart for the Web... 984ms
✓ Built build/web
Flutter server started with PID 17675
Setting up watches. Beware: since -r was given, this may take a while!
Watches established.
```

Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...

6. Launch the application

- open the Skills Network Toolbox in the left bar of the Cloud IDE.
- click on Launch Application button
- enter 8080 as the port
- select the launch in new window to open the application in a new tab

7. Open developer tools to see the mobile view and look at local storage.

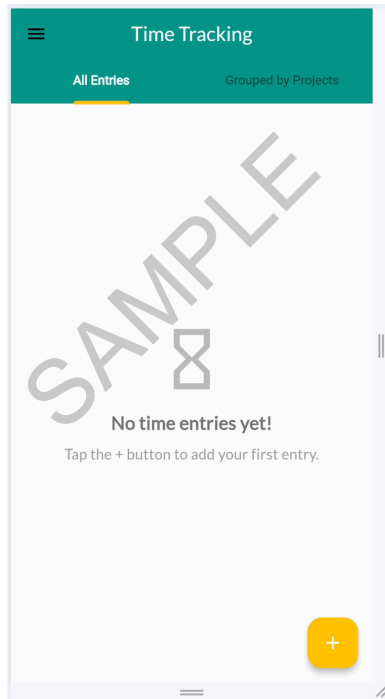
Screenshots for the peer review

Assessment:

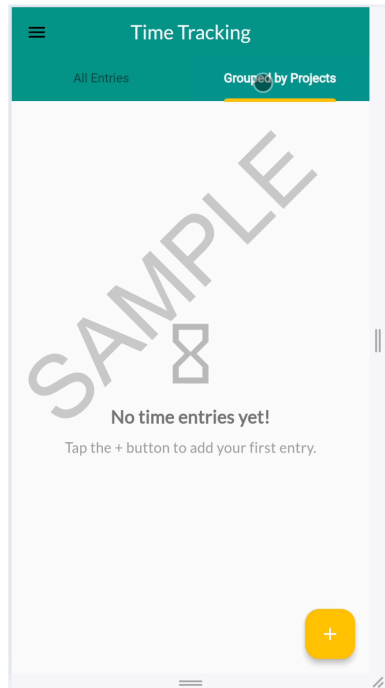
Option 2: Peer-Graded Submission and Evaluation:

Take the following screenshots for peer review. Make sure you name them as listed here. We have added sample ones for your reference.

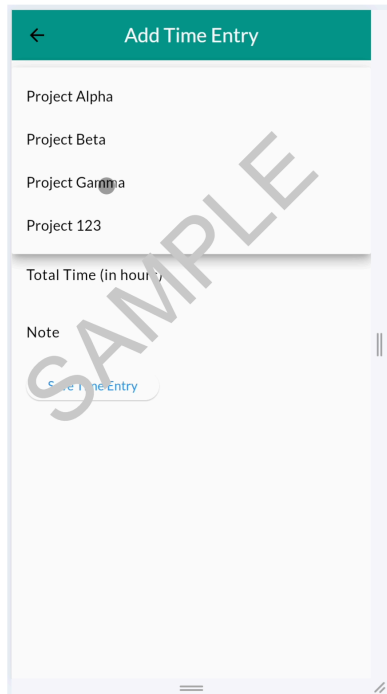
1. home-empty.png(jpeg): A screenshot of the home screen when first launched, showing the empty state with no time entries.



2. home-empty-group.png(jpeg): A screenshot of the home screen with the Projects tab open, showing the empty state with no time entries.



3. entry-project-list.png(jpeg): A screenshot showing the dropdown for projects when adding a new entry.



4. entry-task-list.png(jpeg): A screenshot showing the dropdown for tasks when adding a new entry.

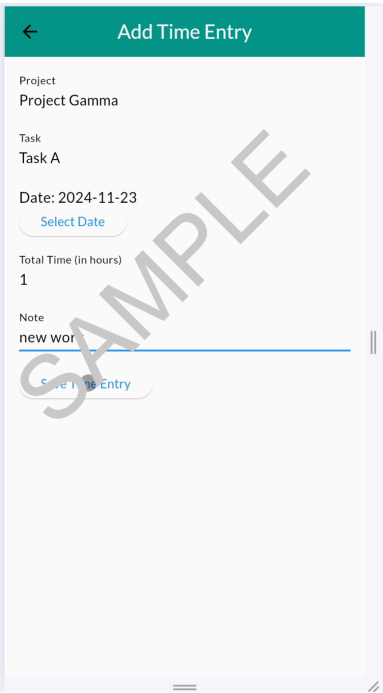
← Add Time Entry

Project
Project Gamma

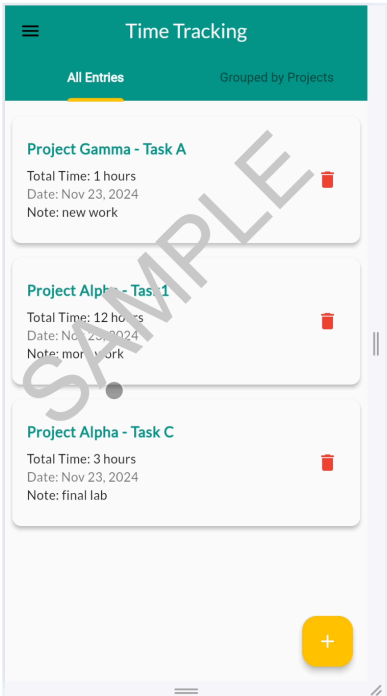
Task A ●
Task B
Task C
Task1
Task 2

Save Time Entry

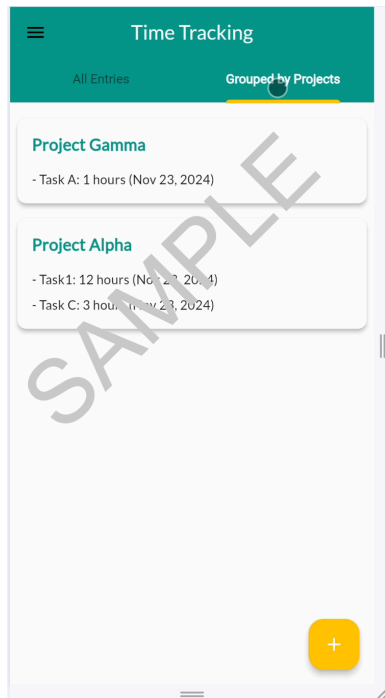
5. entry-add.png(jpeg): A screenshot of the "Add Time Entry" form with fields filled in but before submission, demonstrating the entry process. The fields should be total time, project name, task name, notes, and date.



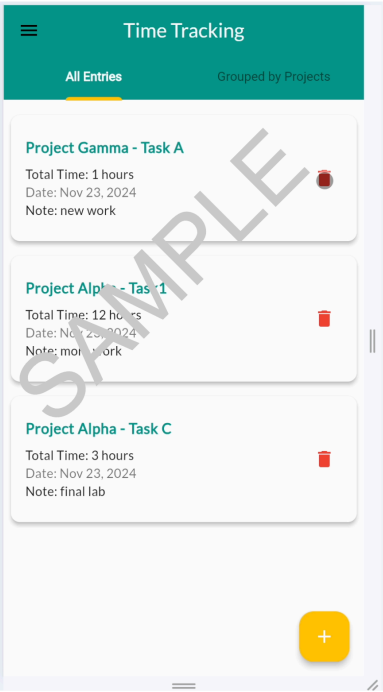
6. home-entries.png(jpeg): A screenshot showing the home screen with multiple time entries listed.



7. home-entries-group.png(jpeg): A screenshot showing the home screen with multiple time entries grouped by projects.



8. entry-delete.png(jpeg): A screenshot of the home screen while deleting an entry, showing the swipe-to-delete action or a delete icon.



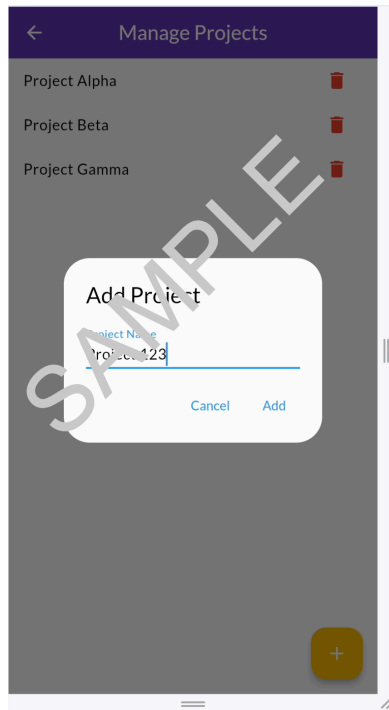
9. menu.png(jpeg): A screenshot of the hamburger menu open showing **Projects** and **Tasks**.



10. project-management.png(jpeg): A screenshot of the project management page showing a list of projects and the floating (+) button to add a new project.



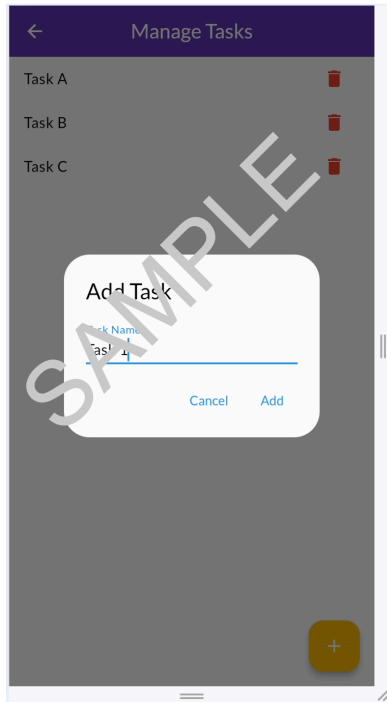
11. project-add.png(jpeg): A screenshot of the user adding a new entry to the list of projects on the management page using a dialog.



12. task-management.png(jpeg): A screenshot of the task management page showing a list of tasks and the floating (+) button to add a new project.



13. task-add.png(jpeg): A screenshot of the user adding a new entry to the list of tasks on the management page using a dialog.



14. local-storage-empty.png(jpeg): A screenshot of the local storage showing an empty list of time entries.



15. local-storage-filled.png(jpeg): A screenshot of the local storage showing a few entries.

http://localhost:63759

Origin http://localhost:63759

Key	Value
projects	[{"id": "1", "name": "Project Alpha", "isDefault": true}, {"id": "2", "name": "Project Beta", "isDefault": false}, {"id": "3", "name": "Project Gamma", "isDefault": false}]
tasks	[{"id": "1", "name": "Task A"}, {"id": "2", "name": "Task B"}, {"id": "3", "name": "Task C"}]
timeEntries	[{"id": "1732356185289", "projectId": "3", "taskId": "1", "start": "2024-11-23 15:32:34.708", "end": "2024-11-23 15:32:34.708"}]

```
▼ [{"id": "1", "name": "Project Alpha", "isDefault": false}, {"id": "2", "name": "Project Beta", "isDefault": false}, {"id": "3", "name": "Project Gamma", "isDefault": false}]
  ► 0: {"id": "1", "name": "Project Alpha", "isDefault": false}
  ► 1: {"id": "2", "name": "Project Beta", "isDefault": false}
  ► 2: {"id": "3", "name": "Project Gamma", "isDefault": false}
  ► 3: {"id": "2024-11-23 15:32:34.708", "name": "Project 123", "isDefault": false}
```

Checklist for submission

Follow the checklist below to verify that your project meets all requirements before submission.

Submit your work through either Option 1: AI-Graded Submission and Evaluation or Option 2: Peer-Graded Submission and Evaluation, depending on the submission path you choose for project evaluation.

Follow the submission checklist below if you are proceeding with **Option 1: AI-Graded Submission and Evaluation:**

1: Submit your **home screen.dart** code showing the following functionality:

- All Entries tab displays empty-state UI when no entries exist.
- Grouped by Projects tab displays empty-state UI when no entries exist.
- All Entries tab displays a list of time entries.

- Grouped by Projects tab displays grouped entries.
- Delete entry feature
- Hamburger menu, which includes Projects and Tasks, and navigates correctly.

2: Submit your **add_time_entry_screen.dart** code showing:

- Project dropdown populated from provider data.
- Task dropdown populated from provider data.
- Complete time-entry form with working save functionality.

3: Submit your **project_management_screen.dart** code showing:

- List of projects with delete functionality and the (+) button to add a new project.
- Add project dialog gets called using the (+) button.

4: Submit your **task_management_screen.dart** code showing:

- List of tasks with delete functionality and the (+) button to add a new task
- Add task dialog gets called using the (+) button.

5: Submit your **time_entry_provider.dart** code showing:

- Time-entry list is initialized as an empty list
- Data persistence using LocalStorage for loading and saving entries.

Follow the submission checklist below if you are proceeding with **Option 2: Peer-Graded Submission and Evaluation**:

1. A screenshot of the home screen when first launched, showing the empty state with no time entries, saved as **home-empty.png(.jpeg)**
2. A screenshot of the home screen with the Projects tab open, showing the empty state with no time entries, saved as **home-empty-group.png(.jpeg)**
3. A screenshot showing the dropdown for projects when adding a new entry, saved as **entry-project-list.png(.jpeg)**
4. A screenshot showing the dropdown for tasks when adding a new entry, saved as **entry-task-list.png(.jpeg)**
5. A screenshot of the "Add Time Entry" form with fields filled in but before submission, demonstrating the entry process (total time, project name, task name, notes, and date), saved as **entry-add.png(.jpeg)**
6. A screenshot showing the home screen with multiple time entries listed, saved as **home-entries.png(.jpeg)**
7. A screenshot showing the home screen with multiple time entries grouped by projects, saved as **home-entries-group.png(.jpeg)**
8. A screenshot of the home screen while deleting an entry, showing the swipe-to-delete action or a delete icon, saved as **entry-delete.png(.jpeg)**

9. A screenshot of the hamburger menu open showing Projects and Tasks, saved as **menu.png(.jpeg)**
10. A screenshot of the project management page showing a list of projects and the floating (+) button to add a new project, saved as **project-management.png(.jpeg)**
11. A screenshot of the user adding a new entry to the list of projects on the management page using a dialog, saved as **project-add.png(.jpeg)**
12. A screenshot of the task management page showing a list of tasks and the floating (+) button to add a new project, saved as **task-management.png(.jpeg)**
13. A screenshot of the user adding a new entry to the list of tasks on the management page using a dialog, saved as **task-add.png(.jpeg)**
14. A screenshot of the local storage showing an empty list of time entries, saved as **local-storage-empty.png(.jpeg)**
15. A screenshot of the local storage showing a few entries, saved as **local-storage-filled.png(.jpeg)**

Conclusion and next steps

This final project is focused on creating a basic time tracking app. In the upcoming steps, you'll enhance the app's functionality to ensure that the time entries are not only captured but also organized and manageable.

Next steps

Congratulations on completing the time tracking application! Here are some advanced features and improvements you can implement to further enhance your application:

- **Implement sorting and filtering**
 - Add functionality to sort time entries by date, duration, or project
 - Implement filters to view time entries by specific projects, tasks, or date ranges
- **Add data visualization**
 - Implement charts to visualize time spent on different projects or tasks over selected periods
 - Use a package like `charts_flutter` to create bar charts, pie charts, or line charts that provide insights into time management
- **Enhance UI/UX**
 - Improve the overall user interface and user experience
 - Add animations, transitions, and better visual feedback for interactions like adding or removing time entries

- **Leverage APIs**

- Integrate project management APIs to allow users to import projects and tasks directly from their existing project management tools
- Enhance functionality to export time tracking data to formats compatible with other tools or systems

- **Implement advanced time management features**

- Add support for starting, stopping, and pausing timers, which can help in logging time more accurately for various tasks

By implementing these advanced features, you will create a more robust and user-friendly time tracking application that can help users effectively manage their time. Good luck, and happy coding!

Author(s)

Skills Network

© IBM Corporation. All rights reserved.